

ソフトウェアデザイン 2026

学生向け演習

第2～15回

情報システム学科 3年生 選択2単位

授業で案内する提出先・期限に従って使用する。

演習のフォルダ構成は本教材の推奨例。

第02回 自習演習：Pythonの基礎

この資料は、値を使う小さなプログラムから、注文を集計する関数までを自分で確かめるための教材です。第1～17節の基礎のコード例は独立しています。前の例を実行していなくても、そのコード枠だけで試せます。第18節以降の注文集計は、自分でsummary関数を実装してから検証します。

読み方と実行の準備

Python 3.12以上とテキストエディタを用意します。ターミナルで `python3 --version` を実行し、使っているPythonの版を確認してください。3.12未満なら、第01回の環境案内に戻り、利用するPythonを切り替えます。

リポジトリのルートで作業用ファイルを用意します。

```
mkdir -p work/02
```

各節のPythonコードを `work/02/practice.py` に保存し、次のコマンドで実行します。例を切り替えるときはファイルの内容を置き換えます。コード枠の外に示す出力や説明は、Pythonファイルへ貼り付けません。

```
python3 work/02/practice.py
```

毎回、次の順で進めてください。

1. 実行前に、出力またはエラーを予測する。
2. コードを実行し、予測と結果を比べる。
3. 違った場合は、どの行で値・処理順・型が変わったかを書く。
4. 確認問題を考えてから、その下の理由つき解答を読む。

予測が外れることは、理解を確かめる材料です。初めから答えを合わせる必要はありません。記録は「入力・予測・結果・理由」の4項目で十分です。

1. 値と表示

```
print(200)
print("200")
print(200 + 120)
```

出力：1行目が 200、2行目も 200、3行目が 320。

200 は整数の値、"200" は文字列の値です。画面では同じ形に見えても、できる操作は異なります。200 + 120 は式で、計算すると値320になります。print は渡された値を画面に表示します。文字列の囲みの引用符は表示しません。

確認：print("200" + "120") の出力は？

答えと理由：200120。文字列どうしの + は連結なので、数の加算にはなりません。

2. 代入と更新

```
price = 200
original = price
price = price + 50
print(price)
print(original)
```

出力：250、200。

= は右辺を評価し、得たオブジェクトに左辺の名前を結び付ける代入です。price = price + 50 は、更新前のpriceを使って250を計算してから、priceをその結果へ結び直します。originalの結び付きは変更していません。

確認：original = price は、今後もpriceの変更に追従する計算式を保存している？

答えと理由：いいえ。その時点でpriceが指す整数200へoriginalも結び付きます。後の代入はpriceという名前の結び付きを変更するので、originalは200のままです。リスト自体を変更する例は第9節で扱います。

3. 型と変換

```
text = "3"
count = int(text)
print(type(text))
print(type(count))
print(count + 2)
```

出力：<class 'str'>、<class 'int'>、5。typeの表示では、引用符内のstrやintが型の名前です。

型は、その値をどのように扱うかを決めます。よく使う型は、整数int、浮動小数点数float、文字列str、真偽値boolです。int("3") は文字列から整数への変換。元のtextは文字列のままです。変換は、任意の文字列で成功するとは限りません。

確認：int("ノート") は？

答えと理由：ValueError。intは文字列を整数の表記として解釈しますが、この文字列は整数を表していません。input() で受け取る値も文字列なので、数として使うときは変換と不正入力の扱いが必要です。

4. 演算と優先順位

```
print(2 + 3 * 4)
print((2 + 3) * 4)
print(7 / 2)
print(7 // 2)
print(7 % 2)
```

出力：14、20、3.5、3、1。

掛け算は加算より先に評価します。括弧を使うとまとまりを指定できます。/ は通常の除算、// は床除算、% は剰余です。この正の整数の例では、7は「2を3回と、余り1」に分けられます。// は負数も扱う床除算なので、常に小数部分を0の方向へ切り捨てる説明にはしません。

確認：6 / 2 の値と型は？

答えと理由：値は 3.0、型はfloat。この整数同士の / もfloatを返します。注文課題で整数の単価・数量を使うときは、除算が必要かを先に考えます。

5. 比較と条件

```
quantity = 3
print(quantity == 3)
print(quantity > 3)
print(0 <= quantity < 10)
print(quantity != 0 and quantity < 5)
```

出力：True、False、True、True。

`==` は等しいかを比較し、`!=` は等しくないかを比較します。比較の結果は真偽値です。`and` を使うと、この例では両方の比較が成立する条件を作れます。`=` の代入とは使い方が異なります。

確認：quantityを0に変えた場合、最後の行は？

答えと理由：False。quantity `!= 0` が成立しないためです。and は左辺が偽と判断できれば右辺を評価しません。Pythonのand自体は真偽値への変換結果ではなくオペランドを返しますが、ここでは比較式同士なので結果もboolです。

6. ifとインデント

```
quantity = 0
if quantity > 0:
    print("注文あり")
else:
    print("注文なし")
print("確認終了")
```

出力：注文なし、確認終了。

条件が成立すればifの内側、成立しなければelseの内側を実行します。行頭の字下げでブロックを表します。最後のprintは字下げがないので、どちらの分岐でも実行します。字下げにはスペース4個を揃えて使いましょう。

確認：最後のprintをelseと同じ字下げにすると、quantityが2のときに何が変わる？

答えと理由：確認終了 を表示しなくなります。最後のprintもelseのブロックへ入り、quantityが2ではそのブロックを実行しないためです。見た目の整形だけでなく、実行範囲が変わります。

7. forで合計する

```
prices = [200, 120, 80]
total = 0
for price in prices:
    total = total + price
    print(total)
print("合計", total)
```

出力：200、320、400、合計 400。

forはリストから要素を順に取り出し、そのたびにpriceへ結び付けて内側を実行します。totalは、その時点までの合計を保ちます。

取り出す値	加算前のtotal	加算後のtotal
200	0	200
120	200	320
80	320	400

確認：prices = [] にすると、最後の行は？

答えと理由：合計 0。要素がないのでforの内側を1度も実行せず、初期値0を保ちます。入力の順序を変えても、整数の同じ値を合計するこの例の最終値は同じです。重複した値は、それぞれの要素として加算します。

8. rangeとwhile

回数を指定するforの例：

```
for i in range(3):
    print(i)
```

出力：0、1、2。range(3) は0から始まり、終了値3を含みません。

条件を確かめ続けるwhileの例：

```
count = 0
while count < 3:
    print(count)
    count = count + 1
print("終了", count)
```

出力：0、1、2、終了 3。条件を各回の先頭で評価します。countが3になると条件が偽になり、whileを抜けます。

確認：countの更新を削ると？

答えと理由：countが0のままなので、条件がずっと成立し、0を繰り返し表示します。実行した場合はターミナルでControl+Cを押して止めます。forの要素の切り替えと違い、このwhileでは自分で終了へ近づく値を更新する必要があります。

9. リストの位置と変更

```
prices = [200, 120, 80]
print(prices[0])
print(prices[-1])
prices[1] = 150
prices.append(50)
print(prices)
```

出力：200、80、[200, 150, 80, 50]。

リストは順序を持つ値の集まり。位置を指定するインデックスは0から始まります。負のインデックスは末尾から数え、-1が最後の要素です。要素への代入やappendは、既存のリストを変更します。

確認：長さ3の変更前のリストで prices[3] を読むと？

答えと理由：IndexError。使える非負インデックスは0、1、2です。len(prices) は要素数3ですが、最後の位置は len(prices) - 1 です。空リストには末尾の要素也没有せん。

10. 同じリストを指す名前

```
prices = [200, 120]
alias = prices
alias[0] = 999
print(prices)
print(alias is prices)
```

出力：[999, 120]、True。

aliasとpricesは同じリストを指します。aliasから要素を変更すると、その1つのリストの内容が変わります。is は同じオブジェクトかを確かめる比較です。== は、このリスト同士では内容の等しさを比較します。

確認：alias = [10, 20] と結び直した後も、pricesがその新しいリストになる？

答えと理由：いいえ。aliasという名前の結び付きだけが変わります。「同じリストの要素を変える」と「名前を別のリストへ結び直す」を区別します。

11. 辞書のキー

```
order = {"name": "ノート", "price": 200}
print(order["name"])
print("quantity" in order)
print(order.get("quantity", 0))
```

出力：ノート、False、0。

辞書は、キーと値の対応を持ちます。order["name"] はnameというキーに対応する値を読みます。辞書に対するinは、キーが存在するかを調べます。getは、キーがないときの既定値を指定できます。

確認：order["quantity"] を読むと？ また、注文課題で個数をgetの既定値0にしてよい？

答えと理由：この例では KeyError。課題ではquantityが必須なので、不足を0とみなして受け入れてはいけません。キー不足を検出し、仕様どおりValueErrorとして伝えます。getを使う場合も、不足と有効な値の区別を設計します。

12. 浅いコピー

```
prices = [200, 120]
copied = prices.copy()
copied[0] = 999
print(prices)
print(copied)
print(copied is prices)
```

出力：[200, 120]、[999, 120]、False。

copyは外側の新しいリストを作ります。copiedの0番を別の整数へ置き換えても、pricesの0番は変わりません。ただし、内側にリストや辞書がある場合、その内側まで新しくする処理ではありません。

```
orders = [{"quantity": 2}]
copied = orders.copy()
copied[0]["quantity"] = 9
print(orders)
```

出力：[{'quantity': 9}]。新しい外側のリストも、同じ内側の辞書を指しています。

確認：注文の入力を変えない関数なら、最初に浅いコピーを作れば十分？

答えと理由：コピー後に内側の辞書を変更するなら十分ではありません。今回の集計は、入力を読むだけにし、合計を別の整数として計算できます。

13. 関数の引数と戻り値

```
def subtotal(price, quantity):
    return price * quantity

amount = subtotal(200, 2)
print(amount)
print(subtotal(120, 3))
```

出力：400、360。

defは関数を定義します。呼び出しの200と2が引数としてpriceとquantityへ渡ります。returnは計算した値を呼び出し元へ返し、その関数呼び出しを終了します。同じ処理へ別の値を渡せます。

確認：returnの直後に、同じ字下げで print("ここ") を置くと？

答えと理由：そのreturnを実行した呼び出しでは表示しません。returnで関数を抜け、後ろの行へ進まないためです。

14. printとreturn

```
def show_total(price, quantity):  
    print(price * quantity)  
  
result = show_total(200, 2)  
print(result)
```

出力：400、None。

最初の400は関数内の表示です。この関数はreturnで値を指定していないため、呼び出し元へNoneを返します。画面への出力と、計算に再利用する戻り値は、別の役割です。

確認：この後 result + 10 を計算すると？

答えと理由：TypeError。resultは整数400ではなくNoneなので、整数10との加算が定義されていません。注文のsummaryは総額を「表示する関数」ではなく、総額の整数を「返す関数」として作ります。

15. 関数内の名前とスコープ

```
quantity = 2  
  
def next_quantity(quantity):  
    quantity = quantity + 1  
    return quantity  
  
print(next_quantity(quantity))  
print(quantity)
```

出力：3、2。

スコープは、名前を参照できる範囲に関係します。関数の引数quantityは、関数呼び出し内のローカルな名前です。その名前を3へ結び直しても、外側のquantityの結び付きは変わりません。

確認：関数なら、リストや辞書を渡しても入力は絶対に変わらない？

答えと理由：いいえ。渡されたリストを関数内でappendしたり、辞書の要素を変更したりすれば、呼び出し元からも見えます。ローカル名の結び直しと、共有するオブジェクト自体の変更は別です。入力非変更は、関数という形だけでは保証されません。

16. エラーの読み方

例は1つつファイルへ保存して実行します。途中で例外が出ると、処理がその場所で止まるためです。

例	エラー	確認すること
<code>print(unknown)</code>	<code>NameError</code>	名前の定義・綴り
<code>print("3" + 2)</code>	<code>TypeError</code>	演算する値の型
<code>int("abc")</code>	<code>ValueError</code>	変換できる値か
<code>[10, 20][2]</code>	<code>IndexError</code>	位置が範囲内か
<code>{"price": 200}["quantity"]</code>	<code>KeyError</code>	キーが存在するか

構文自体をPythonが解釈できない場合は`SyntaxError`、字下げの構造が不正なら`IndentationError`などになります。実行中の例外と、実行前の構文の誤りを区別しましょう。

エラーメッセージは、最後の行で種類と説明を読み、自分のファイル名と行番号が示す箇所へ戻ります。その行に使った値を確認します。型が違った場合は、なぜその型になったかも前の行へ追います。

確認：エラー行を削って実行が終われば、修正完了？

答えと理由：必要な処理を削っただけかもしれません。期待する値・仕様を満たしたかを、同じ入力で再実行して確かめます。

17. 入力検証と例外

```
def checked_quantity(value):
    if type(value) is not int or value < 0:
        raise ValueError("quantityは0以上の整数です")
    return value

print(checked_quantity(0))
print(checked_quantity(True))
```

結果：最初に 0 を表示し、次の呼び出しでValueErrorを送出します。エラーで実行が止まることが、この例の期待する結果です。

0は有効な個数です。Trueはboolであり、個数としては受け入れない契約です。Pythonのboolはintの派生なので、`isinstance(True, int)` はTrueになります。ここでは `type(value) is int` によって整数型を厳密に要求します。短絡評価により、型が違えば `value < 0` を実行しません。

確認：`def checked_quantity(value: int)` と型注釈だけを付ければ、文字列やboolを拒否できる？

答えと理由：できません。型注釈だけでは、Pythonは実行時の入力検証を自動実行しません。仕様に反する値はifで判定してraiseするなど、実際の処理が必要です。assertは自分の検証に使えますが、利用者の入力検証にはifとraiseを使います。

18. 注文集計へ組み合わせる

小さな例で確かめた知識を、次の関数へまとめます。最初に仕様と期待値を読み、自分で実装してから公開解説例と比較してください。

`summary(orders) -> int` の仕様：

- 入力全体はリスト、各要素は辞書で、`name・price・quantity`を必須とする。
- `name`は文字列。空文字もこの仕様では許す。
- `price`と`quantity`は、0以上の厳密な整数型。bool・小数・数の文字列を拒否する。
- 戻り値は円単位の総額の整数。注文なしは0。
- キー不足、不正な型、負数は、理由が分かるValueErrorを送出する。
- 入力のリストや辞書を変更しない。同じ名前の注文も各行の単価×個数を加算する。

受入例：

```
orders = [  
    {"name": "ノート", "price": 200, "quantity": 2},  
    {"name": "ペン", "price": 120, "quantity": 3},  
]  
assert summary(orders) == 760  
assert summary([]) == 0
```

確認：760円になる理由と、空の入力が0になる理由は？

答えと理由：200×2と120×3の和は400+360=760。合計を0から始め、注文ごとに加算する設計なら、空の入力では加算せず0を返せます。

実装は、合計の初期化、forで各注文を読む処理、必須項目・型・範囲の検証、小計の加算、ループ後のreturnへ分けて考えます。returnをループ内に置いた場合、最初の1件しか集計しないので、2件以上の入力で確認します。

19. 結果と入力非変更を確かめる

正常、空、数量0、負数、文字列、小数、bool、キー不足、入力全体の型違い、要素の型違いを試します。期待値は実装とは独立に、先に手で求めます。異常入力はValueErrorを期待するので、単に「実行が失敗した」とだけ記録しません。

入力非変更を確かめる例。summaryを定義したファイルの末尾などで実行します。from copy import deepcopy は、Pythonの標準ライブラリからコピー用の関数を読み込む記述です。

```
from copy import deepcopy  
  
orders = [{"name": "ノート", "price": 200, "quantity": 2}]  
before = deepcopy(orders)  
assert summary(orders) == 400  
assert orders == before
```

確認：beforeを before = orders と保存しても、同じ検証になる？

答えと理由：なりません。同じリストを指すため、関数が入力を変更するとbeforeから同じ変更が見え、等しいという比較が通ってしまいます。この検証では、入れ子の辞書を含めて別に保存するdeepcopyを使います。集計関数本体にdeepcopyが必要という意味ではありません。

解説例の実行：

```
python3 examples/02-basics/order_summary.py
```

既定の自作データでは総額760円、品目別数量はノート2、ペン3。JSON出力の `total_yen` が総額です。公開例の `summary` は整数だけを返し、`summarize` は品目別数量も含む辞書を返します。2つの戻り値を混同しないでください。

最後の説明チェック

資料を閉じて、次を具体的な入力で説明してみましょう。

- `=` と `==` は何をするか。
- `for`の2回目に、`total`はいくつになるか。
- 空入力でどの行を実行し、どの行を実行しないか。
- `return`を`print`へ変えると、呼び出し元で何が変わるか。
- なぜ`bool`を明示的に拒否し、入力の辞書を変更しないのか。

AIを使う場合は、答えを受け取って終わらず、状態の説明や反例を依頼し、自分でも実行してください。「予測・実際の結果・理由」の記録に、支援を使った箇所と自分で検証した箇所を添えると、学習を振り返れます。

概念デモでも確かめる

`concepts.py` は、指定したCASEを1つ実行する補助デモです。本文の短い例とは入力や表示の書式が異なる場合があるので、数値の完全一致だけでなく、そのCASEのコードと状態の変化を照合してください。

```
python3 examples/02-basics/concepts.py for-trace
```

このCASEの期待出力：

```
initial total=0
step=1, price=200, before=0, after=200
step=2, price=120, before=200, after=320
step=3, price=80, before=320, after=400
empty list: iterations=0, total=0
```

stepは反復の番号、beforeとafterは加算前後の値です。空入力は反復0回で合計0になります。別のCASEを試すときは、コマンドの最後を次の名前へ置き換えます。

CASE	主に確かめる内容
rebinding	整数の再代入と文字列の操作
types	型、演算、浮動小数点数
conversion	文字列から数への変換
conditions	比較と分岐
lists	インデックス、共有、コピー
shallow-copy	外側のコピーと内側の共有
dictionary	キーでの読み書き
for-trace	合計の変化と空入力
range	開始・終了・増分
while-update	更新と終了条件
functions	戻り値、表示、ローカル名
exceptions	例外の種類と原因

exceptionsは、意図した例外をデモ内で捕捉して表示するので、終了コードは0です。ValueErrorのtracebackは、実際に得たファイル名・関数名・コード行の抜粋で、絶対パスと行番号を省略しています。通常のエラー全文を読む例では、自分のファイルの行番号も確認してください。

提出課題の案内

正式な提出先・期限・参照条件は授業時の案内に従います。教材の推奨構成は submission/02/summary.py、test_summary.py、短いREADME（実行方法）とreflection（予測・修正・AI支援の要約）です。テストは標準ライブラリのunittestで作成できます。

```
python3 submission/02/summary.py
python3 -m unittest discover -s submission/02 -v
```

参考

- [Python 3.12公式：入門](#)
- [Python 3.12公式：制御構文と関数](#)
- [Python 3.12公式：データ構造](#)
- [Python 3.12公式：エラーと例外](#)

本文の例・注文データ・確認問題は本教材用の自作です。

第3回 AI時代の学習と開発

到達目標

生成AIを学習と開発に使い、採用の判断を反例・実行結果・説明で支える。Python 3.12以上、エディタ、第2回の集計コードを用意する。サービス固有の有料契約は教材の必須条件ではない。

A 学習の依頼

説明しづらいPythonの概念を1つ選ぶ。自分の予測・疑問を書き、AIに小さな例や質問を依頼する。回答と異なる入力を自分で作り、手元で実行する。自分の説明がどのように変わったか200字程度で残す。

B 二分探索の診断

examples/03-ai-workflow/binary_search_buggy.pyは検証教材として自作した誤りの例。整列済み整数リストから一致する添字を返し、不在ならNoneという仕様。重複は任意の一致を認める。未整列入力は対象外。

1. [1] から1を探す場合を追跡し、失敗を再現する。
2. バグの原因候補と、候補を区別するテストをAIに尋ねる。
3. 修正を submission/03/search.py に作る。完成例をコピーする前に、探索範囲が閉区間か半开区間かを決める。
4. 空・1要素・末尾・不在・重複の5種類をテストする。重複例の期待値は「返した添字の値がtargetと一致する」でもよい。
5. verify.py と修正版を参照し、自分の判断を比較する。

```
python3 examples/03-ai-workflow/verify.py
python3 -m unittest discover -s submission/03 -v
```

C 相互説明

相手が選んだ入力に対する結果を、実行前に予測する。境界の更新と終了条件を説明し、実行後に一致を確認する。本人のコードの仕様を質問する活動であり、他人の提出物をAIへ送る必要はない。

提出物

search.py、test_search.py、README.md、reflection.md。記録には依頼の要点、採用/却下した候補と理由、期待値の求め方、実行結果、残っている前提を含める。提出先・期限は授業の案内に従う。

AIを使える環境がない場合は、スライドの提案例をAIの候補と見立て、相互質問で同じ検証手順を行う。環境状況と実際に行った手順を記録する。個人情報・秘密情報を依頼に含めない。

確認問題

説明の流暢さと正しさの違い、1要素の反例の意味、AIが作る期待値の検証、却下した提案の記録を、それぞれ自分の作業に結び付けて説明する。

第04回 演習：処理順と増え方

学生用。Python 3.12以上の環境で、リポジトリのルートから実行する。python3 --version で利用中の版を確認する。正式な提出先・期限は授業時の案内に従う。以下は本教材の推奨成果物。

準備

この回は多くの構造・探索法を概観する。当日は、スタック・キューと木の処理順、二分探索の候補区間を深く追う。増え方とメモリは小さいモデルから説明する。ソート全方式、Nodeの定義、ジェネレーターの自力実装は追加の課題ではない。以下の既存の段階1〜3と成果物を、順に確かめる。

work/04 を作り、記録用の 04-observations.md を置く。

```
mkdir -p work/04
python3 examples/04-algorithms/containers.py
python3 examples/04-algorithms/trees.py
```

まず実行せずに、A、B、Cをこの順に追加したときのスタックとキューの取り出し順を予測する。木はAの子がB、C、Bの子がD、E、Cの子がF。子の列挙順を保ってDFSとBFSの訪問順を予測する。

検証の基準は、スタックがC B A、キューがA B C、木のDFSがA B D E C F、BFSがA B C D E Fとなること。実演出力と照合し、異なる場合は取り出し方法と子の順を調べる。

木の公開例を読む足場

lectures/04.md の「訪問した名前をリストに残す」では、同じリストへ名前を追記し、訪問が全部終わってから return する。trees.py もこの方式。辞書の children[name] が、公開版では node.children に対応する。Node は名前 value と子のリスト children を一組にした節点で、node.value はその名前を読む記法である。

スライドの yield 版は別の返し方を読む補足。yield は1つの名前を渡し、yield from は子から出る名前を順に渡す。list(...) で最後まで取り出せば訪問順をリストにできる。公開版の dfs(demo_tree()) は初めからリストを返すので、yield を理解していなくても予測と照合を進められる。

段階1：探索を実装する

work/04/04-search.py に、線形探索と二分探索を作る。どちらも一致する要素のインデックスを1つ返し、不在なら None を返す。二分探索の入力は昇順に整列済みであること。重複の先頭を返すことは、この段階の要件に含めない。

入力 [8, 2, 6, 4] と値6を使う。線形探索の期待値は2。二分探索には整列したコピー [2, 4, 6, 8] を渡し、期待値は2。このインデックスは整列後の列に対応する。元の列の位置に戻りたい場合は、追加の設計が必要になる。

比較の前後で候補を読む

スライドは半开区間 [lo, hi)、sorting_search.py と第3回は閉区間 [low, high]。自分の実装はどちらでもよいが、初期値・継続条件・更新を一組でそろえる。以下は既存の「存在しない値」の検証を読むための例で、新しい提出課題ではない。

[2, 4, 6, 8] で5を探す。半开区間では右端を含まない。

比較前のlo, hi	中央	次の区間になる理由
0, 4	添字2の6	5は6より小さい。hi = 2で中央も除く
0, 2	添字1の4	5は4より大きい。lo = 2へ進める
2, 2	候補なし	lo < hiが偽なのでNone

閉区間は両端を含むため、同じ入力で次のようになる。

比較前のlow, high	中央	次の区間になる理由
0, 3	添字1の4	low = 2で中央の次へ進める
2, 3	添字2の6	high = 1で中央の前まで残す
2, 1	候補なし	low <= highが偽なのでNone

1要素の [6] では、半开区間は (lo, hi) = (0, 1)、閉区間は (low, high) = (0, 0) で始まる。どちらも比較が必要。閉区間の low == high を候補なしと取り違えると、第3回の見落としが再発する。

次の条件も自分で追加する。

- 空列で不在を返す
- 先頭と末尾の要素を見つける
- 存在しない値を調べる
- 1要素の列で一致・不一致を調べる

```
python3 work/04/04-search.py
python3 examples/04-algorithms/sorting_search.py
```

参考デモのソート入力は [7, 2, 4, 2, 9]、整列後は [2, 2, 4, 7, 9]。自分の入力の結果と混同しない。

段階2：候補数を数える

3要素の列で、異なる2要素を選ぶ候補をすべて列挙する。 $i < j$ として、同じ組の重複を避ける。候補数は3。4要素なら6、5要素なら10になることを確認する。

「全探索」という名前だけで計算量を決めず、候補の数と、候補1件の確認コストを説明する。巨大な列を作らず、小さい入力で増え方を確かめる。

段階3：メモリと操作数を読む

```
python3 examples/04-algorithms/memory_complexity.py
```

出力から入力サイズと操作数モデルの値を記録する。これは実際の処理時間・比較回数を測定した値ではなく、増え方を比較する計算モデルである。入力を2倍にしたとき、線形の操作数はおよそ2倍、二重ループの操作数は4倍という関係を説明する。デモが用いるサイズ・操作の定義をコードで確認する。

メモリのbyte数には、Pythonの版・処理系・環境による差がある。`sys.getsizeof` が直接測るオブジェクトと、その参照先を区別する。全データのbyte数と断定しない。

提出用の成果物

- 04-search.py：実装と検証用入力
- 04-observations.md：予測、実際の結果、探索の前提、候補数とメモリの説明

記録にはPythonの版と実行コマンドを添える。AIを使った場合は、質問の要約・採用した提案・自分で確認した入力を書く。未使用なら、その旨を書く。最後に、資料を閉じて二分探索の区間が毎回狭くなる理由を説明する。

自分で確かめる問い

1. 未整列入力に二分探索を適用する危険を、具体例で示せるか。
2. BFSで `pop()` を使うと処理順はどう変わるか。
3. リストの浅いコピーは何をコピーするか。
4. 実行時間1回の測定だけで、計算量を断定できるか。

発展として、重複があるときの「最初の一致位置」を返す二分探索を考えてよい。必須範囲を検証した後に進む。

第05回 演習：グラフの設計と追跡

学生用。Python 3.12以上を使う。正式な提出方法は授業時の案内に従う。以下は本教材の推奨手順。

準備と実演

```
mkdir -p work/05
python3 examples/05-graphs/graphs.py
```

参考デモはA-B、A-C、B-D、C-Dを持つ無向グラフと、孤立頂点X。AからDのBFS経路は['A', 'B', 'D']。連結成分は2つ。出力と図を照合する。

段階1：架空校内マップを作る

次の自作の場所をグラフにする。入口A、受付B、図書室C、演習室D、別棟X。直接行き来できる通路はA-B、A-C、B-D、C-D。Xには通路がない。通路は両方向に通れ、各通路を移動1回として数える。

work/05/05-map.py に、全頂点をキーに持つ隣接リストを書く。Xの値は空のリスト。接続を出力し、図と照合する。無向辺は両方の頂点の隣接リストへ記録する。

期待する性質は、Aの隣がBとC、Dの隣がBとC、Xの隣が空であること。無向グラフの次数は、単純グラフであれば隣接頂点の数。Aの次数は2、Xは0。

段階2：BFSを手で追う

AからDを探す。Aの隣接順はB、Cとする。頂点をキューへ入れた時点で発見済みにする。開始時点を含め、取り出し1回ごとに次を記録する。

- 取り出した頂点
- 処理後に残るキュー
- 新しく追加した親の記録

Aを取り出した直後のキューはB、C。Bを処理した直後はC、D。Cを処理しても、Dを再追加しない。Dの親はBのまま。この状態からA B Dを復元できるか確認する。

段階3：入力を変える

マップのコピーでB-Dを両方向から削除する。AからDへの最少辺数は2のままだが、経路はA C Dになる。さらにC-Dも両方向から削ると、DはAから到達不能になる。元の入力に戻せるよう、変更前後のデータを分けて保存する。

到達可能な頂点を集合として出力する場合、元のグラフでAからの集合はA、B、C、D。XからはXだけ。

段階4：有向グラフへの変更

一方通行の仮想通路として、AからB、BからDだけを持つデータを別に作る。AからDには到達できるが、DからAには到達できないことを確認する。

Aの出次数は1、入次数は0。Bの出次数・入次数は両方1。次数の名前を正負の値と混同しない。

移動時間を扱う重み付きグラフへ変えるなら、BFSが最短時間を保証できる条件を 05-design.md に書く。重み付き最短経路の実装はこの回の必須範囲ではない。

提出用の成果物と検証

- 05-map.py：無向・有向の自作データと出力
- 05-design.md：頂点・辺・向き・距離の意味、BFS追跡、変更後の期待値と結果

```
python3 work/05/05-map.py
```

AIを使った範囲と、自分で照合した辺・親の記録を書く。未使用なら未使用と書く。次回に向け、キューと親辞書の役割を説明する文を残す。

自分で確かめる問い

1. 無向辺を片側だけ登録すると、どんな問題が起こるか。
2. Aから到達できる集合だけで、有向グラフ全体の強連結性を判断できるか。
3. BFSで複数の最短経路のうち1本を返す場合、何が返す経路を決めるか。
4. 孤立頂点を入力に明示する理由は何か。

第06回 演習：最少移動数の経路を実装する

学生用課題。データ構造とアルゴリズムの知識を確認し、授業内で実装・提出を行う。

Python 3.12以上。正式な提出先・締切・参照条件は授業時の案内に従う。以下の成果物構成は本教材の推奨案であり、正式な点数配分は定めない。

授業内の進め方

作業コピーと最初の失敗を確認したら、探索・復元の実装を始める。講義の段階2～5は、必要な処理へ戻る参照として使う。実装の後に、関数とCLIの検証、成果物整理、提出完了の確認を行う時間を残す。既存の受入条件と成果物案は変更しない。

課題の仕様

架空駅の無向・重みなしグラフを使う。駅はA、B、C、D、E、X。辺はA-B、B-D、A-C、C-E、E-D。Xは孤立駅。

`find_route(graph, start, goal)` は、最少辺数の経路を頂点名のリストで1本返す。既知の駅同士が到達不能なら `None`。未知の駅名、または無向辺として不整合なデータには `ValueError`。既存の入力検証を保ち、仕様の条件を壊さないこと。

同じ駅を指定した場合は `[start]`。辺数は `len(path) - 1`。複数の最短経路がある入力では、1本の妥当な最短経路を返せばよい。このデータのAからDの最短経路はA B D。

CLIは出発駅と到着駅を引数に取る。省略時はA D。成功は終了コード0。到達不能と未知の駅名は終了コード2で、理由がわかるメッセージを表示する。既存のCLIを利用してよい。

準備

リポジトリのルートから実行する。

```
python3 --version
mkdir -p work/06
cp examples/06-route-project/route_starter.py work/06/route.py
cp examples/06-route-project/route_data.py work/06/route_data.py
python3 work/06/route.py A A
python3 work/06/route.py A B
python3 work/06/route.py A D
```

学生版は同駅と隣駅のみを処理する。A Dは多段経路が未実装なので失敗する。この時点の出力を記録し、実装すべき範囲を確認する。work/06/route.py の find_route を編集する。

段階1：設計を先に書く

README.md に、最短の定義、入力的前提、戻り値、終了条件を書く。キュー、親辞書、訪問済み判定の役割を説明する。未知の駅と孤立駅Xを区別する。

段階2：BFSを実装する

出発駅をFIFOのキューに入れ、親辞書に start: None を記録する。取り出した頂点の隣を調べ、未発見なら親を記録してキューに入れる。到達したgoalから親をたどり、経路を反転して返す。

同じ頂点を重複してキューに入れないこと。到達不能のgoalについて、親の復元を始めないこと。実装の各段階で、小さい入力を使って状態を追う。

段階3：必須ケースを検証する

入力	関数の期待値	CLIで確認すること
A D	['A', 'B', 'D']	移動数2、正常終了
D A	['D', 'B', 'A']	移動数2、正常終了
A A	['A']	移動数0、正常終了
A X	None	到達不能、終了コード2
UNKNOWN D	ValueError	入力エラー、終了コード2

隣駅A Bも確認する。自作の小さい入力を1件追加し、手で求めた最短経路と比較する。

```
python3 work/06/route.py A D
python3 work/06/route.py D A
python3 work/06/route.py A A
python3 work/06/route.py A X
python3 work/06/route.py UNKNOWN D
```

CLIのエラー後に終了コードを確かめる場合、直後に `echo $?` を実行する。エラー表示は、この2ケースでは想定する結果である。

段階4：関数として確かめる

`work/06/check_route.py` を作る。単純なassertでもよいので、CLI表示と関数の結果を分けて確認する。

```
from route import find_route
from route_data import GRAPH

assert find_route(GRAPH, "A", "D") == ["A", "B", "D"]
assert find_route(GRAPH, "A", "A") == ["A"]
assert find_route(GRAPH, "A", "X") is None
path = find_route(GRAPH, "D", "A")
assert path[0] == "D" and path[-1] == "A"
assert all(b in GRAPH[a] for a, b in zip(path, path[1:]))
print("基本ケースを確認")
```

```
python3 work/06/check_route.py
```

期待出力は「基本ケースを確認」。未知の駅の例外検証や、自作入力の検証も自分で加える。上の確認だけですべての仕様を検証したことにはならない。

公開参考実装

`examples/06-route-project/route_reference.py` は解説・照合用の公開参考実装。先に自分の設計と結果を残す。参照した場合は範囲と学んだ内容を記録し、授業時に案内する参照条件に従う。参考版を自分の実装と偽って提出しない。

提出前の確認と成果物

`work/06` に次をまとめ、案内された提出方法で授業内に提出する。提出を完了したことも確認する。

- `route.py`、`route_data.py`、`check_route.py`
- `README.md`：実行方法、Python版、仕様、設計
- `test-results.md`：入力、期待値、実際の結果、終了コード
- `reflection.md`：AIの支援範囲、提案の変更、自分で検証した内容、残る疑問

未完成なら、実行できる範囲と問題の再現入力を明示する。第07回に向け、自分のキューと親辞書がA Dでどう変化するかを説明できる状態にする。

第07回 演習：講評と質疑

学生用。共通のレビューには本教材が自作した架空提出例を使う。実学生の提出物・評価・個人情報を含めない。自分の第06回の成果物を用意する。正式な再提出の扱いは授業時の案内に従う。

段階1：架空提出例を実行する

この回のグラフは第06回のデータと異なる。無向・重みなしの辺はA-B、B-E、E-F、F-D、A-C、C-D。孤立頂点Xを含む。

```
mkdir -p work/07
python3 examples/07-review/flawed_route.py A D
python3 examples/07-review/improved_route.py A D
```

架空提出例は訪問済み管理したDFS。最初に得た経路A B E F Dを「最短」と表示する。移動数4。改善版はBFSでA C Dを返し、移動数2。この違いを図と出力で照合する。

work/07/07-review.md に、満たしている仕様、満たしていない仕様、反例、改善後に確認する条件を書く。経路が存在する点まで誤りと決め付けない。

段階2：テストを追加する

同駅A A、孤立駅A X、未知駅UNKNOWN Dを両方のCLIで試す。成功時の経路・辺数と、失敗時のメッセージ・終了コードを記録する。XからXは同駅なので経路X、移動数0となることも確認する。

発展として、作業コピーのグラフからA-Cの辺を両方向に削り、最少経路がA B E F D、4辺になるか検証する。その結果でも、元のDFSが一般に最短を保証するとは言えない理由を書く。共有の参考ファイルは変更しない。

段階3：改善の根拠を書く

「dequeを使ったから」だけで終えず、FIFOで近い層が先に処理され、未発見の頂点の親を最初の発見時に記録することを説明する。重みが異なるグラフなら、最少辺数と最少時間が一致するとは限らない。

講評文には、対象の入力、実際の結果、期待する振る舞い、改善後の確認方法を含める。能力や人格への評価を避け、コードの振る舞いを扱う。

段階4：自分の提出物を説明する

第06回のコードを開き、次の問いへ、該当箇所と具体的な入力を使って答える。

1. この関数が返す「最短」は、何を最少にした経路か。
2. キューと親辞書は、A Dの探索中にどう変化するか。
3. 同じ頂点がキューへ何度も入らない根拠はどこか。
4. 到達不能と未知の駅名をどう区別したか。
5. 自作テストは、どの実装ミスを検出する狙いで選んだか。
6. AIの提案を採用した箇所について、自分が確認したことは何か。

担当教員との授業内の質疑で説明する。説明に詰まった場合は、入力を小さくして状態を書き出す。質疑で気付いた疑問・修正点を残す。

段階5：改善を検証して記録する

work/07/07-improvements.md に、第06回の修正前後の振る舞い、再現入力、期待値、実際の結果を書く。修正が不要なら、確認した条件とその証拠を記す。AIを使用したか、何に使い、何を自分で検証したかを振り返る。

成果物の推奨構成は 07-review.md と 07-improvements.md。実装も修正した場合は、修正版コードとテスト結果を一緒に整理する。提出・再提出するかは授業時の案内に従う。

次回への準備

電卓の入力文字列をどう区切るかを考える。例は $12 + 3 * 4$ 。数と演算子を識別し、演算のまとまりを木で表す場面を予想する。次回は字句解析・構文解析・意味解析の区別を学ぶ。

第8回 電卓の解析と実行

到達目標

字句・構文・意味の境界を、トークン、AST、テストで説明する。Python 3.12以上、木の走査と関数を復習する。公開referenceはexamples/08-calculator/calculator.py、動作する拡張用starterはstarter.py。Pythonのeval/execを使わない。

当日の中心と進め方

まずAのトークン、Bの木と評価を通して、 $2+3*4$ の入力位置・規則の呼出し・部分ASTをつなぐ。Cでは、既存の「完成基礎電卓を利用して拡張する」「自作で全実装する」のどちらで進めるかを決める。全処理系を白紙から書く作業と、解析の途中状態を説明する学習を同じ量と考える。以下の受入例・提出物は両経路で確認する。

コマンドはリポジトリのルートから実行する。完成基礎電卓は動く状態を観察する入口であり、実行しただけでは本人の拡張実装を示したことにはならない。

```
python3 examples/08-calculator/starter.py
python3 examples/08-calculator/calculator.py --eval '2+3*4'
```

starterはASTの表現と結果14.0を表示し、calculatorのCLIは14を表示する。どちらも同じ数値を計算している。自分の変更・テストへ進む前に、小さい入力動くことを確認する。

A トークンの追跡

$12+3.5*(2-1)$ を種類・文字列・開始位置に分け、末尾eofも記録する。空白を挿入した場合、意味は同じで開始位置が変わることを確かめる。`2@3` を受理せず、`@`の位置を説明する。

B ASTと評価

$2+3*4$ 、 $(2+3)*4$ 、 $10-3-2$ の木を描く。期待値は14、20、5。子の評価が終わるごとに、途中結果を書き込む。

木ができる途中を読む補足

公開版の Parser は、トークン列と次に読む添字 pos を持つオブジェクト。オブジェクトは関連する情報・操作をまとめたもの。parser.expression() の内部の self は、そのparser自身を指す。self.current は現在のトークンを見るだけで位置を進めず、self.take() はそのトークンを返してposを1増やす。別の規則を呼んでも、同じ位置を引き継ぐ。

2+3*4 のトークンの添字は 2:0、+:1、3:2、*:3、4:4、eof:5。ここでのposは文字列の位置ではなくトークン列の添字である。規則がどこまで読んで何を返すかを、次の順で見る。

読む規則と動き	pos	できている木
expression → term → factorが2を読む	0 → 1	Number(2.0)。termは+を読まず返す
expressionが+を読む	1 → 2	左の2の木を保持し、右のtermを呼ぶ
右のterm → factorが3を読む	2 → 3	右側の左の木はNumber(3.0)
termが*、factorが4を読む	3 → 5	Binary("...", Number(3.0), Number(4.0))
expressionが左右をつなぐ	5のまま	足し算を根に持つ木。parseがeofを確認

Number は数値の葉、Binary は演算子・左の子・右の子を持つ木の部品。ここでtermが返すのは3×4の木で、数値12ではない。evaluate が木を受け取った後に3×4を12、2+12を14と評価する。詳細は lectures/08.md の「途中1」～「途中3」、公開解説は instructor/08.md。この途中例はBの理解を助けるもので、新しい提出課題ではない。

C 実装または拡張

submission/08に電卓を作り、字句・構文・評価を別関数に分ける。参照版はスライドのEBNF全体に対応する。学生演習の基本必須は整数表記と 3.5 のように小数点の前後に数字がある表記。3.、.5 は発展として実装してもよい。対応範囲をREADMEに明示し、基本例は全て満たす。

starter.py は基礎の電卓を呼ぶ動作例。拡張課題は、独立の検査・表示・テストを増やすこと。自力実装する場合も完成例と比較してよい。

経路	開始状態と本人の作業
完成基礎電卓を利用して拡張	動く字句・構文・評価を利用し、submission/08に本人の検査・表示・テストなどを作る。利用元と変更範囲をREADMEで示す
自作で全実装	submission/08に字句・構文・評価を別関数で作し、各段階の結果を完成例と比較して検証する

いずれもA・Bの説明と以下の受入例を用いる。完成参照を利用した範囲と、本人が実装して確認した範囲を区別してreflectionに書く。

受入例（正常な入力）：

入力	期待値または失敗
2+3*4	14
(2+3)*4	20
-3*2	-6
8/2/2	2
0.1+0.2	誤差を考慮して0.3に近い

受入例（拒否する入力）：

入力	期待する失敗
2@3	字句エラー
(2+3	構文エラー
2 3	余分なトークンとして構文エラー
1/0	評価エラー

```
python3 examples/08-calculator/calculator.py --eval '2+3*4'
python3 -m unittest discover -s submission/08 -v
```

エラーメッセージを出しただけで合格にせず、該当段階をテストする。例を手計算してから実行する。

提出物

実装、テスト、README（文法と実行方法）、AST図、reflection（AI採用判断、失敗と修正、段階の説明）をまとめる。提出先・期限は授業の案内に従う。

確認

$2+3$ は読めるトークンだが並びが不正。ASTは空白そのものより意味を持つ構造を残す。
10-3-2は左結合。ゼロ除算は構文として合法的式の評価条件である。自分の処理系で各理由を説明する。

第9回演習 LispのREPLと四則演算

教材と作業用コード

PythonホストとLisp方言の仕様は本教材の設計上の選択であり、正式シラバスの指定言語仕様ではない。

examples/09-lisp-arithmetic/lisp.py は実行可能な完成参照。starter.py は既存機能を使いながら 既存の square 組込みへ入力検証を追加するウォームアップ用の実行可能 starterで、講義の全機能が未実装の穴埋めではない。ウォームアップでは作業用に複製した starter.py の machine.add_builtin にあるlambdaを名前付き関数へ置き換え、数値検証の後で負数なら LispError を送出する。正常入力と負数の反例を確かめる。square は meta側へ自動登録されない。

本課題は前回の自作版、または別名の作業用コードで進める。完成参照を使った範囲を明記し、自分の実装と混同しない。比較のために参照版をそのまま残す。

演習手順と成果物は教材案。公式の提出先、締切、点数配分をここでは規定しない。

到達目標

S式の読取と四則演算の評価を分け、失敗後も次の入力を受けるREPLを作る。

始める前に

第8回のトークン、木、再帰の説明を読み直す。Python 3.12以上を使い、プロジェクトのルートから実行する。第9回のホストREPLは1行1式で扱う。複数行の式入力はこの段階の受入条件に含めない。

段階1 読取

(+ 1 (* 2 3)) のトークン列と木を手で書く。読取関数へ渡し、数値・記号・入れ子リストの区別を確認する。閉じ括弧不足と余分な閉じ括弧をそれぞれ入力し、評価へ進む前に失敗することを確認する。

段階2 評価

前回の自分のコードまたは新しい作業用ファイルに、数値と二項の +、-、*、/ を実装する。通常の手出しは引数を評価してから計算する。単項演算や可変長引数を追加する場合

は、対応仕様を明記する。完成参照の + と * は0個以上、- と / は1個以上。単項減算は符号反転、単項除算は逆数。複数引数の減算と除算は左から順に計算する。まず二項の版を通し、参照の契約へ段階的に広げる。

段階3 REPL

入力、読取、評価、表示を循環する。1式の失敗を報告した後、次の式を受ける。:quit と EOFを終了として扱う。診断文の完全一致より、失敗原因を識別でき、循環が継続することを確認する。

受入例

入力	期待
(+ 2 3)	5
(- 9 4)、(* 3 4)	5、12
(/ 8 2)	数値として 4
(* (+ 2 3) (- 9 4))	25

失敗例は (/ 1 0)、(+ 1 2、(+ 1 2))、(unknown 1 2)。失敗後に (* 4 5) を入力して 20 を得る。読み取りと評価のどちらで失敗したか説明する。

検証コマンド

```
python3 examples/09-lisp-arithmetic/lisp.py --eval "(+ 1 (* 2 3))"
python3 examples/09-lisp-arithmetic/lisp.py
```

参照版の結果は 7。2行目のREPLで正常、エラー、正常の順に入力する。自分の版は同じ入力と比較する。この回はファイル実行を課題にせず、第11回で追加する。

成果物案

- 自分の読取・評価・REPLのコードと実行コマンド
- トークン列と木を示す説明
- 受入例、失敗例、回復例のログ

- 対応する数値・引数個数・表示の仕様

AI利用の振り返り

AIに求めた説明または修正案、採用した理由、自分で実行した反例を記録する。生成されたコードを説明できない箇所は、関数を小さく分けて読取と評価を追う。

確認問い

(+ 1 2) で + を先頭に置く意味は何か。括弧不足とゼロ除算を同じ診断にすると、誰が何を困るか。

第10回演習 変数と関数の定義

到達目標

環境による名前検索、`define`、`lambda`、`if`、`quote` を実装し、クロージャと再帰を説明する。

授業内の中心と継続作業

当日は `define`、単純な `square` の適用、非選択枝を評価しない `if` を順に確認する。クロージャでは `make-adder` の4状態、再帰では `fact 2` の下降と復帰を図と表で説明する。自作の全機能を当日白紙から完成する前提は置かず、未完の機能は確認できた地点から継続して実装する。

以下は授業内の進め方の整理。提出課題の仕様、受入例、成果物、配点は従来どおり。クロージャや再帰の到達点を削除したり、補足driverを新たな提出条件にしたりしない。

教材と作業用コード

PythonホストとLisp方言は教材の設計上の選択であり、正式シラバスの指定言語仕様ではない。`examples/10-lisp-functions/lisp.py` は完成参照。基本は第9回の自作版を複製して進め、参照を使った部分を本人の実装と区別する。

`starter.py` は既存 `square` 組込みへ入力検証を追加するウォームアップ用で、本課題全体の穴埋めではない。当日の中心を終えてから扱える。行う場合は作業用コピーの `machine.add_builtin` にある `lambda` を名前付き関数へ置き換え、数値検証後、負数なら `LispError` を送出する。`square` は meta 側へ自動登録されない。公式の提出先、締切、点数配分をここでは規定しない。

開始コードの確認

第9回の自分の版を作業用に複製する。数値・記号・リストを区別できることを確認する。文字列やファイル実行は第11回の範囲。複数の式を連続実行する参照検証には `begin` を使える。

段階1 環境の定義

評価関数に環境を渡す。記号は現在の環境から外側へ探し、見つからなければ診断する。
`define` は名前の式を評価せず、右辺だけを評価して登録する。定義の返り値は自分の仕様に書き、定義後の名前の値を受入条件にする。

段階2 関数

`lambda` は仮引数、本文、定義時環境を保存する値を作る。呼出し時には実引数を評価し、新しい環境へ束縛する。新しい環境の外側は関数の保存環境とする。引数個数が合わない場合を、束縛前に診断する。

段階3 特殊形式

`if` は条件と選択した枝だけを評価する。教材方言では `#f` が偽で、数値 `0` や空リストも真として扱う。`quote` は受け取った式を評価せず返す。通常関数適用とは別の分岐にする。

段階4 再帰とクロージャ

```
(define make-adder
  (lambda (x) (lambda (y) (+ x y))))
(define add5 (make-adder 5))
(add5 3)
```

上の改行は説明用。教材REPLは1行ごとに1式を読むため、各 `define` のまとまりを1行にして同じセッションへ入力し、`8`を確認する。次のCLIなら改行を含む定義を1個の `begin` として検証できる。

```
python3 examples/10-lisp-functions/lisp.py --eval '
(begin
  (define make-adder
    (lambda (x) (lambda (y) (+ x y))))
  (define add5 (make-adder 5))
  (add5 3))'
```

講義の4状態図に対応して、次を追う。`G・E5・E3`と`C0・C5`は説明用のラベルであり、自作コードへ同じ名前を追加する条件ではない。

状態	作られた値・環境	確かめること
make-adderの定義	関数C0がGを保存	本文はまだ評価しない
make-adder 5の呼出し	x=5のE5、内側の関数C5	C5がE5を保存する
外側から復帰	Gのadd5がC5を参照	呼出しが終わってもE5へ参照が残る
add5 3の呼出し	y=3のE3、外側はE5	yはE3、xはE5、+はGで検索し8を返す

呼出しが実行中であることと、環境への参照が残ることを分ける。add7 を別に作る反例で、異なるxの束縛が混ざらないか確かめる。未完の自作機能は完成参照で理解を確認し、差分を継続実装へ残す。

```
(define fact
  (lambda (n)
    (if (= n 0) 1
        (* n (fact (- n 1))))))
(fact 4)
```

こちらの define fact も、REPLには定義全体を1行にして入力する。fact 0 は 1、fact 4 は 24。まず小さな fact 2 で戻る過程を追う。

戻る呼出し	残っていた計算	戻り値
fact 0	停止条件の1	1
fact 1	(* 1 1)	1
fact 2	(* 2 1)	2

下降はn=2、1、0、復帰はn=0、1、2。各呼出しに別のnの束縛があり、外側には同じfactの定義がある。負数など未対応の入力を、検証済みの範囲と区別する。

受入例

入力	期待
(begin (define x (+ 2 3)) x)	5
(begin (define square (lambda (x) (* x x))) (square 6))	36
(if #f (/ 1 0) 7)	7、ゼロ除算なし
(quote (+ 1 2))	リスト (+ 1 2)

追加の反例として未定義名、square の引数個数違いを記録する。外側の $x = 100$ と引数 $x = 4$ を共存させ、呼出し後も外側が 100 のままであることを確認する。

検証コマンド

```
python3 examples/10-lisp-functions/lisp.py --eval '(if #f (/ 1 0) 7)'
python3 examples/10-lisp-functions/lisp.py
```

補足観察と公開解説

```
python3 examples/10-lisp-functions/learning_trace.py closure
python3 examples/10-lisp-functions/learning_trace.py fact
```

補足driverは既存の完成ホストで、make-adderの定義、E5の生成、捕捉環境、E3の適用と結果8、fact 2の下降と復帰を観察する。自分の環境図と比較するための資料であり、新しい提出条件やfixtureではない。参照の成功を自作版の完成と混同しない。

説明の根拠は、講義の「1 make-adderの定義」～「4 add5の呼出し」と「再帰の下降」「再帰の復帰」。理由の補足は [公開教員ノート](#) を読める。

成果物案

実装コード、環境図、受入例と反例のログ、特殊形式の評価順序の説明を残す。定義時と呼出し時の環境を別々に描く。

AI利用の振り返り

AIに環境探索を説明させ、自分の図と比較する。実行前の予測、反例での観察、修正した説明を保存する。AIが動的スコープを提案した場合は `make-adder` など検討する。

確認問い

`lambda` の本文を定義時に評価すると何が起きるか。 `if` に両方の枝の値を先に渡してはならない理由は何か。

第11回演習 スクリプト実行と評価器の準備

教材と作業用コード

PythonホストとLisp方言の仕様は本教材の設計上の選択であり、正式シラバスの指定言語仕様ではない。

examples/11-lisp-scripts/lisp.py は実行可能な完成参照。starter.py は既存機能を使いながら 既存の square 組込みへ入力検証を追加するウォームアップ用の実行可能starterで、講義の全機能が未実装の穴埋めではない。ウォームアップでは作業用に複製した starter.py の machine.add_builtin にあるlambdaを名前付き関数へ置き換え、数値検証の後で負数なら LispError を送出する。正常入力と負数の反例を確かめる。square は meta側へ自動登録されない。

本課題は前回の自作版、または別名の作業用コードで進める。完成参照を使った範囲を明記し、自分の実装と混同しない。比較のために参照版をそのまま残す。

演習手順と成果物は教材案。公式の提出先、締切、点数配分をここでは規定しない。

到達目標

UTF-8のファイルから複数式を順に実行し、Lisp製評価器に必要なリスト・型・入出力操作を確かめる。

当日の作業経路

最初に段階1の「複数式を同じ環境で順に評価する」経路を通す。文字列・コメント・リストの説明は、その入口を検証するために読む。第10回の自作版に必要な操作がまだない場合は、完成ホストで同じ入力を観察してから自作版との不足を特定する。完成ホストで動いたことを、自作版の達成とは扱わない。

文字列・コメントのlexer改修とlist操作の追加は、自作版の開始状態によって継続作業になる。当日の主経路と追加改修を分けるための案であり、以下の段階・受入例・成果物の契約は変更しない。授業内の進行は開始時の実装状況に合わせる。

段階1 スクリプト入口

第10回の自分の版にファイル実行の入口を加える。ファイル全体をUTF-8で読み、終端まで式を繰り返し読む。同じ環境で順に評価する。ファイルを開けない失敗、読取時の失敗、

評価時の失敗を区別する。スクリプトでの失敗後の方針はREADMEに記述する。参照版は実行を停止して非ゼロ終了する。

作業用 `lesson11.lisp` に次を保存する。

```
(define square (lambda (x) (* x x)))
(display (square 5))
(newline)
(display (square 7))
(newline)
```

表示は 25 と 49。参照版のCLIは最後の式の値だけを自動表示する。この例では末尾の `newline` の返り値 `()` も最後に表示する。途中の値を表示するには `display` と `newline` を使う。

同じ5個の式を次のように追う。

評価の位置	共有する大域環境	見える出力
defineの後	squareの定義を保持	まだ表示しない
最初の呼出しの後	同じsquareを保持、局所x=5は呼出し内	25
次の呼出しの後	同じsquareを保持、局所x=7は別の呼出し内	49

毎回大域環境を作り直す実装では、最初の`display`が`square`を見つけられない。参照版の局所引数環境と、式間で共有する環境を区別する。

段階2 字句の状態

"hello world"、"a;b"、行コメント `;` memo を試す。文字列中の空白と `;` を失わないことを確かめる。記号と文字列を内部の型で区別する。括弧を文字列内に含めた例も追加する。

例 `(display "a (b); c") ; memo` で、次の状態を追う。図と同じ名前のstate変数を必ず作る必要はない。参照実装の`tokenize`は、文字列を読む内側のループと行末への移動でこの区別を表す。

入力部分	状態・処理
(display	外側で括弧と記号を確定
開く引用符	文字列内へ入り、蓄積を開始
a (b); c	空白・括弧・セミコロンを文字列に蓄積
閉じる引用符、)	文字列を確定し、外側で括弧を読む
; memo、改行	コメントを捨て、次の行は外側から読む

結果のトークンは開き括弧、Symbol(display)、文字列 a (b); c、閉じ括弧。読取後に記号と文字列の型が別であることも確認する。エスケープした引用符は文字列を閉じず、閉じない文字列は読取エラーになる。状態の詳しい図は lectures/11.md、理由と支援の順序は公開補足 instructor/11.md にある。

段階3 リスト操作

list、car、cdr、cons、null? を使い、空・1要素・複数要素のリストを調べる。空リストから先頭を取る失敗も記録する。

```
(define total
  (lambda (xs)
    (if (null? xs) 0
        (+ (car xs) (total (cdr xs))))))
(display (total (quote (2 3 4))))
(newline)
```

結果は 9。total に空リストを渡すと 0。同じ例の途中状態は次のとおり。

呼出して縮むxs	その呼出しが待つ式	子から戻る値	この呼出しの値
(2 3 4)	(+ 2 ...)	7	9
(3 4)	(+ 3 ...)	4	7
(4)	(+ 4 ...)	0	4
()	null?が真なので0	子を呼ばない	0

表の下まで呼び、下から上へ戻る。空のxsではcarを呼ばない。教材のcdrは残りを新しいリストとして返し、元のxsを削除しない。(cons 5 (cdr (quote (10 20 30))))は(5 20 30)、元の引用データは(10 20 30)のまま。

段階4 セルフホスティングの準備

Lispの関数へ (quote (+ 1 2)) を渡し、car と cdr で式を分解する。数値・記号・リストを判定する操作を試す。次回の評価器で必要になる操作を、対象の式と対応付ける。読取やOS入出力をどの層に任せるかも記述する。

受入例

入力・状況	期待
上の lesson11.lisp	25、49、最後の値 () を改行して表示
(car (quote (10 20)))	10
(cdr (quote (10 20)))	(20)
(cons 5 (quote (10 20)))	(5 10 20)

加えて、存在しないファイル、括弧不足のファイル、未定義名のファイルを検証する。失敗の段階を説明し、終了状態も記録する。

検証コマンド

```
python3 examples/11-lisp-scripts/lisp.py lesson11.lisp
python3 examples/11-lisp-scripts/lisp.py --eval '(null? (quote ()))'
```

2行目の結果は #t。作業用ファイルを保存した場所から実行し、別の場所で行う場合はパスを合わせる。

成果物案

自分の実装、作業用スクリプト、実行コマンド、正常・失敗ログ、評価器に必要な操作の一覧を残す。新しい操作を追加する場合は、入力と期待する型も記述する。

AI利用の振り返り

AIの提案する文字列の字句解析を、空白・コメント記号・括弧の反例で確かめる。採用したコードと、検証していない入力を区別して記録する。

確認問い

ファイルを空白だけで分割すると、どの入力 that 壊れるか。式ごとに環境を初期化すると、スクリプトの何ができなくなるか。

第12回演習 Lisp製評価器の最小実装

教材と作業用コード

PythonホストとLisp方言の仕様は本教材の設計上の選択であり、正式シラバスの指定言語仕様ではない。

`examples/12-meta-evaluator/lisp.py` は実行可能な完成参照。`starter.py` は既存機能を使いながら 既存の `square` 組込みへ入力検証を追加するウォームアップ用の実行可能starterで、講義の全機能が未実装の穴埋めではない。ウォームアップでは作業用に複製した `starter.py` の `machine.add_builtin` にある`lambda`を名前付き関数へ置き換え、数値検証の後で負数なら `LispError` を送出する。正常入力と負数の反例を確かめる。`square` は meta側へ自動登録されない。

本課題は前回の自作版、または別名の作業用コードで進める。完成参照を使った範囲を明記し、自分の実装と混同しない。比較のために参照版をそのまま残す。

`evaluator_starter.lisp` は自己評価される数値・真偽値・文字列だけを扱う実行可能な最小starter。`starter-eval 42` の値を確認してから、引用と算術を段階的に追加する。`evaluator.lisp` は完成参照。第14回以降の `repl.lisp` も完成参照である。

演習手順と成果物は教材案。公式の提出先、締切、点数配分をここでは規定しない。

到達目標

ホストと対象言語を分け、Lispで書いた最小評価器を動かす。必要な機能の契約を、具体的な対象式で裏付ける。

当日実装・契約設計・後続の区別

扱う段階	既存課題の範囲
当日実装	段階2・3の数値、二項四則、 <code>quote</code> 、 <code>if</code>
契約を設計	段階1・4の名前、環境、対象関数
後続に接続	第13回の説明と反例、第14回の名前・関数・REPL統合

この区別は、既存の段階・受入例・成果物を変えず、実装順序を明確にするためのもの。第12回に対象の名前・クロージャをすべて完成させた前提は置かない。ホストにある `define/lambda` と、自作 `meta` に実装する機能を区別する。

始める前に

第11回の自分の版を使うか、完成ホスト `lisp.py` を基盤としてLisp製評価器を自作する。ホストの不足機能を補う課題と、対象の評価規則を書く課題を混ぜない。基盤として完成ホストを使った場合は、そのことを成果物に明記する。

段階1 機能契約

数値判定、記号判定、リスト分解、環境探索の操作について、入力、結果、失敗条件を記述する。どの対象式を処理するために必要かを添える。Pythonホストの変数と、対象環境の束縛を図で分ける。

段階2 最小評価器

作業用 `my-evaluator.lisp` に、数値をそのまま返す処理を作る。次に `quote` と二項の四則演算を追加する。入力は読取済みS式にする。対象の式を渡すときには、ホストによる先行評価を `quote` で止める。

```
(quote (+ 1 (* 2 3)))
(quote (quote (+ 1 2)))
```

上は対象の計算式をデータとして渡す例。下は対象で引用を処理するためのデータ。2個の `quote` がそれぞれどの層で外れるか説明する。

二重`quote`の例は、次の順でデータになる。

処理する層	式またはデータ	処理後
ホストが外側の <code>quote</code> を処理	(quote (quote (+ 1 2)))	(quote (+ 1 2))
自作 <code>meta</code> が対象の <code>quote</code> を処理	(quote (+ 1 2))	リスト値(+ 1 2)
結果の表示	リスト値(+ 1 2)	値を表示し、再評価しない

対象の入力をexprとすると、carは記号quote、cdrは引数リスト ((+ 1 2))、car(cdr(expr))は値 (+ 1 2)。quote分岐ではこの値をそのまま返す。結果3を得るための再評価は、別の呼出しとして行う操作であり、quoteの受入例には含めない。

完成参照をホストへロードした状態なら、(m-eval (quote (quote (+ 1 2))) (m-global) 0) がこの二層を観察する呼出し。m-evalの引数は対象式、対象環境、深さ。第12回のm-globalは空リストで、envはまだ名前検索に使わない。

(+ 1 (* 2 3)) は内側が6、外側へ渡す引数値が (1 6)、+のprimitive適用が7という順。読み取り済みデータを受け取ることと、内部の引数を評価することは別の処理である。

段階3 分岐

対象の if を追加する。対象条件を評価してから片方の枝だけを対象評価器で評価する。(if #f (/ 1 0) 7) が 7 を返すことを確認する。未対応の対象構文を、ホストへ黙って丸投げせず診断する。

段階4 次の機能の計画

名前検索と対象の define、対象のクロージャへ拡張する契約を設計する。第12回の完成参照のmeta評価器は数値・四則・quote・ifなどの最小段階である。第13回は完成参照も用いて理解を確認し、第14回で対象の関数・環境・REPLの統合へ進む。ホスト側に lambda があることと、対象側に実装できたことを区別する。

名前と関数の契約は、説明用の束縛表 ((x 3) (y 4)) を出発点にする。第14回参照は、そのbindingsを1要素のframe-cellに包み、envを (frame-cell parent) として表す。親のないenvのparentは空リスト。m-bindingsはenvの先頭の先頭、m-lookupは見つからなければ親も探索する。m-defineは共有セルが指す束縛リストを更新し、m-new-envとm-bindが呼出し用の子環境を作る。

この表現は後続参照への橋であり、第12回の最小metaに今すぐ全操作を実装する追加課題ではない。別の環境表現を選ぶ場合も、検索・定義・拡張の入力と結果、見つからない場合の診断をそろえる。途中例は lectures/12.md、解説と支援順序は公開補足 instructor/12.md で読める。

受入例

対象の入力	期待
7	7
(+ 1 (* 2 3))	7
(quote (+ 1 2))	(+ 1 2)
(if #f (/ 1 0) 7)	7、ゼロ除算なし

反例は未知の名前、未知の演算子、引数型の違い。対象側でどこまで診断できるかを書き、未対応機能も明記する。

完成参照の検証

```
python3 examples/12-meta-evaluator/lisp.py --meta --eval '(+ 1 (* 2 3))'
python3 examples/12-meta-evaluator/lisp.py --meta --eval '(if #f (/ 1 0) 7)'
```

どちらも 7。--meta は同じディレクトリの evaluator.lisp を読み、その m-eval を Python製Lispホスト上で動かす。自作版を試すときは、完成参照のファイルを上書きせず、作業用スクリプトとしてホストで実行する。

成果物案

Lisp製最小評価器、必要な操作の契約、ホストと対象を分けた図、受入例と反例のログ、未対応機能と次の開発順序を残す。

AI利用の振り返り

AIの案が対象の名前をホスト環境で検索していないか確認する。提案、確認入力、採用理由または却下理由を記録する。

確認問い

ホストに式を評価させるだけの関数と、対象評価器の実装はどう異なるか。組込みへ委譲した範囲をどの証拠で示せるか。

第13回演習 理解確認とディスカッション

教材と作業用コード

PythonホストとLisp方言の仕様は本教材の設計上の選択であり、正式シラバスの指定言語仕様ではない。

`examples/13-language-review/lisp.py` は実行可能な完成参照。`starter.py` は既存機能を使いながら 既存の `square` 組み込みへ入力検証を追加するウォームアップ用の実行可能 `starter` で、講義の全機能が未実装の穴埋めではない。ウォームアップでは作業用に複製した `starter.py` の `machine.add_builtin` にある `lambda` を名前付き関数へ置き換え、数値検証の後で負数なら `LispError` を送出する。正常入力と負数の反例を確かめる。`square` は `meta` 側へ自動登録されない。

本課題は前回の自作版、または別名の作業用コードで進める。完成参照を使った範囲を明記し、自分の実装と混同しない。比較のために参照版をそのまま残す。

`evaluator_starter.lisp` は自己評価される数値・真偽値・文字列だけを扱う実行可能な最小 `starter`。`starter-eval 42` の値を確認してから、引用と算術を段階的に追加する。`evaluator.lisp` は完成参照。第14回以降の `repl.lisp` も完成参照である。

演習手順と成果物は教材案。公式の提出先、締切、点数配分をここでは規定しない。

到達目標

コードを実行する前に結果を予測し、環境と評価順序を説明する。他者の説明を反例で確かめ、自分の説明を更新する。

今日の進め方

新機能の一斉実装を主活動にしない。個人予測、ペア説明、実行、反証、説明の修正を繰り返す。自分の成功例と未解決例を持ち寄り、互いのコードを尊重して質問する。

授業内は2～3のテーマを選び、予測から説明の更新まで完了する。残る問いは予復習で検討する。議題・受入条件・成果物案は従来どおりで、ここでは授業内の順序を示す。

使う処理系も先に選ぶ。引用と `#f` による分岐は第12回の自作最小 `meta` で試せる。(= 1 1) の比較を含む分岐や、関数・クロージャが未実装なら、対応するホストまたは完成参照を使

い、どの版を使ったかを記す。完成参照で確認した結果を、自作metaの完成機能として数えない。二層の追跡は、付属の完成参照で実行できる。

議題1 引用と分岐

(quote missing)、(if #f (/ 1 0) 8)、(if (= 1 1) 9 missing) の結果を先に書く。次に、評価しない部分へ印を付ける。相手は「通常の関数適用で同じ実装ができるか」と追質問する。実行後、最初の説明との差を記録する。

議題2 クロージャ

```
(begin
  (define make-adder
    (lambda (x) (lambda (y) (+ x y))))
  (define plus2 (make-adder 2))
  (plus2 7))
```

期待する値は 9。x の束縛がどこに残るか図にする。相手は別の加算器を作って、保存した環境が混ざらないか確かめる。動的スコープという説明を受けたら、どの入力で差が出るか検討する。

議題3 環境の分離

外側の $x = 100$ と関数の引数 $x = 4$ を用意する。関数の結果 5 と、呼出し後の外側の値 100 を予測する。単一の変数表を使う架空の案を挙げ、反例で検討する。

議題4 追跡

```
python3 examples/13-language-review/lisp.py --trace --eval '(+ 1 (* 2 3))'
python3 examples/13-language-review/lisp.py --trace --meta --eval '(+ 1 (* 2 3))'
```

結果は両方 7。追跡は標準エラーへ出力する。[host depth=N] はホスト、[meta depth=N] はLisp製評価器の追跡。深さと自分の木を照合する。meta実行中にもホストの処理がある理由を説明する。ログだけでは分からない環境情報がある場合は、追加する証拠を提案する。

議題5 達成範囲

Lisp製評価器が担う処理と、Pythonホストへ委譲する読取・入出力を図にする。対象の quote、if、関数適用がどの層で動くか、関連コードを示す。ネイティブ起動や独立ブートストラップを達成したとは主張しない。

相互質問

相手へ、理解した箇所の問い1個と追質問1個を出す。相手の答えを正誤だけで処理せず、反例や追跡で確かめる。自分が説明できなかった点は、次回の開発課題にする。

受入条件

記録	必要な内容
予測	実行前の値と理由
観察	入力・結果・コマンド
説明	環境図または評価の順序
更新	反例と修正後の説明

未知の小さな式について1件、上記がそろふことを目指す。これは授業用の到達確認であり、点数表ではない。

成果物案

議論記録、相手へ出した問い、環境図、追跡ログ、未解決の問いと次に試す入力を残す。相手の個人情報や提出物を公開せず、自分の記録としてまとめる。

AI利用の振り返り

AIによる実装説明から、検証が必要な主張を1個選ぶ。自分の反例やログで確かめ、説明をどう修正したか書く。AIの説明をそのまま相手への回答にしない。

第14回演習 Lisp製評価器とREPL

到達目標

Lispで書いた対象評価器に、名前、クロージャ、再帰を統合し、Lisp製REPLをPythonホスト上で実行する。継続する環境と、委譲した読取・入出力の範囲を説明する。

授業内の中心と継続作業

当日は同じ `square` を対象式から戻り値まで追い、定義を保つREPLへ接続する。クロージャと再帰の理解確認も行う。名前・関数・REPLの全機能を最小版から一斉に白紙実装する時間は仮定しない。未完の実装は確認できた地点から継続して仕上げる。

以下は授業内の進め方の整理。提出課題の仕様、受入例、成果物、配点は従来どおりで、新しい必須機能や提出物を増やさない。

開始状態と作業の経路

第12回の自作評価器と、第13回の議論記録を用意する。第13回の討論は、対象の名前やクロージャを全員が完成させたことを意味しない。完成ホストでLispスクリプトが動き、自作最小版で数値・引用・四則・分岐のどこまで通るか確かめる。

開始状態	作業の進め方
自作の名前・関数まで動く	自分の評価器を使って対象環境と適用を確認し、REPLへ接続する
最小版まで動く	完成参照で環境と適用を観察する。作業用コピーで不足箇所とREPLの接続を進める

参照版は保存して、別の作業用ディレクトリへコピーする。完成 `lisp.py` をホストとして利用してよい。完成 `evaluator.lisp` を基盤にした場合は、利用した関数と、自分で変更した関数・接続を記録する。参照を実行しただけで、全体を自作したと記述しない。

チェックポイント0 最小式

自作の最小評価器で `42`、`(+ 1 (* 2 3))`、`(quote (+ 1 2))`、`(if #f (/ 1 0) 7)` を確かめる。期待する値は順に `42`、`7`、`(+ 1 2)`、`7`。未対応の構文はそのまま記録する。完成参照の

成功と自作版の成功を分ける。

チェックポイント1 対象環境

講義の「対象環境の形」「対象の名前検索」を使い、現在の束縛と外側を描く。参照の環境は (frame-cell parent)、束縛は (name value)。m-new-env で空の子を作り、m-define で登録し、m-lookup で探す。

外側Gの $x = 5$ と子Eの $y = 3$ を例に、Eで y を見つけ、Eにない x をGで見つける過程を追う。どこにもない名前は診断する。define の右辺を対象評価器で評価してから登録する。ホストの内部名を対象環境で探さない。

チェックポイント2 squareの適用

```
(begin
  (define square (lambda (x) (* x x)))
  (square 6))
```

上の begin は1個の対象式。--meta --eval へ引用した文字列として渡せる。REPLなら定義を1行にして入力する。

順序	確かめる状態
先頭と引数の評価	squareのクロージャと、引数値の列 (6)
新しい対象環境	保存環境Gを外側にしたE6の $x = 6$
本文	E6で (* x x) を評価。* はG、x はE6で検索
復帰	値 36 が呼出し元へ戻る

仮引数と実引数の個数違いも確かめる。ここで失敗したら、REPLの循環を加えずに評価器を直す。

クロージャと再帰の確認

```
(define make-adder
  (lambda (x) (lambda (y) (+ x y))))
(define add5 (make-adder 5))
(add5 3)
```

表示用に改行している。Lisp製REPLへは各定義を1行にして順に入力し、最後の値 8 を得る。保存する $x = 5$ の環境と、呼出し時の $y = 3$ の環境を分ける。第10回の4状態図を対象環境として読み直す。

$\text{fact } 4 = 24$ も既存の受入例で確認する。各呼出しの別の n と、外側で共有する fact を区別する。未完なら完成参照で過程を確認し、自分の版との差を特定して継続実装する。

チェックポイント3 REPL統合

動く対象評価器へ、読取・対象評価・表示の循環を接続する。`m-repl` の対象環境を各入力へ渡し、入力ごとに `m-global` で初期化しない。`read-all` の結果は式の列なので `m-sequence` へ渡す。

参照の `guard` は読取、対象評価、`display`、`newline` を保護する。表示で失敗する場合も次の入力へ戻る。読取・入出力・エラー保護・反復機構はPythonホストへ委譲し、対象の意味はLispの `m-eval` と `m-apply` で決める。

同じセッションで `(define x 5)`、`(+ x 2)`、`(/ 1 0)`、`(+ x 3)` を順に入力する。値と診断は x 、7、ゼロ除算の診断、8。束縛は失敗後も残る。EOFと単独の `:quit` で終了する。教材REPLは1行ごとに読み、同じ行の複数式を扱うが、複数行補完はしない。

教材と作業用コード

PythonホストとLisp方言は教材の設計選択であり、正式シラバスの指定言語仕様ではない。

- `lisp.py` は完成ホスト、`evaluator.lisp` と `repl.lisp` は完成参照。
- `evaluator_starter.lisp` は自己評価する数値・真偽値・文字列だけの最小starter。統合全体の穴埋めではない。
- `starter.py` は既存のsquare組込みを拡張するウォームアップ用。今回の対象評価器やREPLの開始コードと混同しない。squareはmeta側へ自動登録されない。

ウォームアップを行う場合は、作業用 `starter.py` の `machine.add_builtin` にある `lambda` を名前付き関数へ置き換え、数値検証後、負数なら `LispError` を送出する。今回の中心作業を終えてから扱える。公式の提出先、締切、点数配分をここでは規定しない。

受入例

対象の入力	期待
<code>(begin (define square (lambda (x) (* x x))) (square 6))</code>	36
上の <code>make-adder</code> の例	8
<code>(begin (define fact (lambda (n) (if (= n 0) 1 (* n (fact (- n 1)))))) (fact 4))</code>	24
<code>define x=5、(+ x 2)、(/ 1 0)、(+ x 3)</code>	7、診断、 8

最後の行の定義は実際には `(define x 5)` と入力する。反例に未定義名、引数個数違い、非選択枝の失敗を含める。

完成参照の検証と補足観察

```
python3 examples/14-selfhosting/lisp.py --meta --eval '(+ 1 (* 2 3))'
python3 examples/14-selfhosting/lisp.py --selfhost-repl
```

1行目はLisp製評価器の `m-eval` を実行して 7 を得る。2行目は `repl.lisp` によるLisp製REPLを起動する。無引数の `lisp.py` はホストのREPLであり、起動方法の違いを記録する。

補足 `python3 examples/14-selfhosting/checkpoints.py` は、既存の完成参照で最小式、対象環境、関数適用、REPLを段階的に観察する。引数 `minimal`、`environment`、`closure`、`repl` で観察地点を選べる。`closure`では同じ環境操作を使って適用を分解し、通常の手出し結果と比較する。出力を見ただけで自作版の達成としない。新しい提出条件や採点対象を追加するものではない。

成果物案

Lisp製評価器とREPL、ホスト起動コマンド、対象入力と結果のログ、環境図、委譲する操作の一覧、未対応の構文を残す。従来の成果物に、参照を利用した関数と本人の変更箇所を対応付ける。ログと関連コードで、Lisp製評価器を実際に呼んだことを示す。

達成報告は「Python製Lispホスト上で、Lisp製評価器とREPLを実行した」とする。読取器のLisp実装、OSへの直接入出力、ネイティブ実行、独立ブートストラップはこの教材の成果と区別する。

AI利用の振り返り

AIの案がホスト環境と対象環境を混同していないか確かめる。第13回の反例を回帰検証として再実行し、採用した案と結果を記録する。

確認問い

同じ結果が出るだけで、Lisp製評価器を使った証明になるか。エラー後も保持する状態と、捨てる状態は何か。

第15回演習 講評と総括

教材と作業用コード

PythonホストとLisp方言の仕様は本教材の設計上の選択であり、正式シラバスの指定言語仕様ではない。

`examples/15-showcase/lisp.py` は実行可能な完成参照。`starter.py` は既存機能を使いながら既存の `square` 組込みへ入力検証を追加するウォームアップ用の実行可能starterで、講義の全機能が未実装の穴埋めではない。ウォームアップでは作業用に複製した `starter.py` の `machine.add_builtin` にある `lambda` を名前付き関数へ置き換え、数値検証の後で負数なら `LispError` を送出する。正常入力と負数の反例を確かめる。`square` はmeta側へ自動登録されない。

本課題は前回の自作版、または別名の作業用コードで進める。完成参照を使った範囲を明記し、自分の実装と混同しない。比較のために参照版をそのまま残す。

`evaluator_starter.lisp` は自己評価される数値・真偽値・文字列だけを扱う実行可能な最小starter。`starter-eval 42` の値を確認してから、引用と算術を段階的に追加する。`evaluator.lisp` は完成参照。第14回以降の `repl.lisp` も完成参照である。

演習手順と成果物は教材案。公式の提出先、締切、点数配分をここでは規定しない。

到達目標

開発した実装の達成範囲を、仕様・コード・実行結果で説明する。他者の問いを受けて改善案を選び、授業全体を振り返る。

準備

自分の最新コード、仕様、受入例、実行ログを用意する。未完成の箇所も制限として示す。実在の他者の提出物や成績を公開しない。`examples/15-showcase/` は教材用の架空の例であり、学生の提出物ではない。

共通例を確認した後は、ペアで作者と聞き手を交代し、両者が成功例と制限例を説明する時間を確保する。全体へ共有する例は授業で選ぶ。第14回の未完成部分があっても、動く範囲と次の改善を示して参加する。以下の段階・受入条件・成果物案は従来どおり。

段階1 架空の例の講評

講義の例Aは算術だけの実装、例Bは環境を共用し過ぎる設計案、例CはPythonホスト上のLisp製評価器という架空の説明である。付属fixtureは受入入力材料。完成実装の動作と、架空の不具合案の説明を区別する。例Bの誤りは、外側の $x = 100$ と関数引数 $x = 4$ が衝突する入力で検討する。

段階2 自分の実装の紹介

成功例1件について、結果を予測してから実行する。その結果を生む評価規則と環境を図にする。関連するコードの位置を示す。未対応または失敗する入力も1件示し、診断か制限を説明する。

段階3 相互講評

聞き手は作者の説明から曖昧な名前や順序を1つ選び、小さな反例を提案する。作者は結果を予測し、実行して確かめる。観察した事実と改善提案を分けて記録する。意図が不明な箇所は作者へ確認する。

段階4 改善計画

改善を1つ選び、利用者に起きる変化、変更する箇所、受入例を記述する。実装した場合は結果を記録する。計画のみの案を完成済み機能として数えない。

受入条件

成果	確認すること
達成範囲	対応構文と未対応機能を示す
実行証拠	コマンドと再現できる入力がある
講評記録	事実と改善提案を分ける
振り返り	判断が変わった具体例を示す

上記は教材の到達確認案であり、正式な採点配分ではない。

参照の検証

```
python3 examples/15-showcase/lisp.py --meta --eval '(+ 1 (* 2 3))'
python3 examples/15-showcase/lisp.py --selfhost-repl
```

算術の結果は 7。REPLでは自分の紹介例を試す。python3 examples/15-showcase/check_compatibility.py は自作fixture compatibility.json の正常・失敗ケースをホストとmetaで比較する。Lisp製評価器とホストを区別し、読取・入出力の委譲を説明する。

成果物案

自分のコード、仕様、受入例、実行ログ、講評記録、改善計画、全体の振り返りを相互に対応付ける。提出先・締切・正式採点運用は授業で確認する。

全体の振り返り

構文木と環境探索に、前半のデータ構造やアルゴリズムがどう役立ったか、コードの例で説明する。読取、評価、環境、入出力の境界を分けたことで、変更の広がりがどう変わったか振り返る。

AI利用の振り返り

AIへ任せた作業、自分が判断した部分、採用前の検証を具体的に記録する。最初の誤った説明を反例で修正した経験があれば、何を学んだか書く。検証していない主張は未確認と明記する。